Optimisation of OpenMP Cholesky Factorisation on CSD3

MPhil in Data Intensive Science
University of Cambridge

Iuliia Vitiugova

March 2026

List of Figures

3.1 Implementation roadmap: versions V0 to V3 6

7.1 Single thread GFLOP/s vs. n, all versions, icelake 11
7.2 Wall-clock time vs. n (log–log), all versions, icelake 11
7.3 V2 GFLOP/s vs. thread count, icelake 13
7.4 V2 parallel speedup vs. thread count, icelake 14
7.5 V2 speedup heatmap over (n, T), icelake 14
7.6 V2 parallel efficiency vs. thread count, icelake 15
7.7 Measured speedup with fitted Amdahl curves, icelake 15
7.8 Measured speedup with fitted Amdahl curves, cclake 16
7.9 V2 GFLOP/s vs. thread count, cclake 17
7.10 V2 speedup heatmap over (n, T), cclake 18
7.11 V2 GFLOP/s vs. thread count: icelake vs. cclake 19
7.12 Peak GFLOP/s per version, icelake 19
7.13 Peak GFLOP/s per version, cclake 19
7.14 NUMA close/spread ratio vs. thread count, $n = 2000$, cclake 20
7.15 NUMA close vs. spread GFLOP/s, $n = 2000$, cclake 21
7.16 V2 vs. V3 GFLOP/s vs. thread count, icelake 23
7.17 V3/V2 improvement heatmap over (n, T), icelake 23

List of Tables

7.1 Reproducibility metadata for all experiments 8
7.2 GFLOP/s at $T = 1$, all versions, icelake 9
7.3 V2 GFLOP/s vs. thread count, icelake 12
7.4 Amdahl serial-fraction estimates from peak speedup 13
7.5 V2 GFLOP/s vs. thread count, cclake 16
7.6 NUMA close vs. spread GFLOP/s, $n = 2000$, cclake 20
7.7 Same node V2 vs. V3 GFLOP/s comparison, icelake 21

A.1 CMake build targets 26

B.1 Git tagged commits 27

C.1 Test group summary (A-K) 28

D.1 Experiment ledger: result sections to source CSV files 30

Contents

1 Mathematical Introduction **2**
- 1.1 Algorithmic structure (Cholesky–Banachiewicz) . . . 2
- 1.2 Calculation of floating point operations . . . 2
- 1.3 Logarithm of the determinant . . . 3

2 Implementation **3**
- 2.1 Version 0: Serial baseline . . . 3
- 2.2 Version 1: Serial optimised . . . 3
  - 2.2.1 OPT-1: Rearranging the order of loops . . . 3
  - 2.2.2 OPT-2: Hoisting $p{\cdot}n$ and row pointers . . . 4
  - 2.2.3 OPT-3: Replacing division with multiplication . . . 4
  - 2.2.4 OPT-4: Hoisting L_{ip} . . . 4
  - 2.2.5 OPT-5: 4-wide i-unrolling . . . 4
- 2.3 Version 2: OpenMP parallel . . . 5
  - 2.3.1 Analysis of parallelisation and dependencies . . . 5
  - 2.3.2 One persistent parallel region . . . 5
  - 2.3.3 OpenMP constructions . . . 5
  - 2.3.4 Proof of the absence of data races . . . 6
- 2.4 Version 3: Parallelising the column scaling . . . 6

3 Roadmap of Optimisation Versions **6**

4 Test Methodology **7**

6 Benchmark Methodology **7**

7 Results and Discussion **8**
- 7.1 Single thread: $V0 \to V1 \to V2 \to V3$ . . . 8
- 7.2 $V2$ OpenMP scaling (icelake) . . . 12
- 7.3 V2 OpenMP scaling (cclake) . . . 16
- 7.4 Partition comparison and bandwidth saturation . . . 18
- 7.5 Non-Uniform Memory Access experiment . . . 19
- 7.6 Same node V2 vs V3 comparison . . . 21

8 Conclusions **23**

Bibliography **25**

A Repository Structure **26**

B Tagged Commits **27**

C Build and Tests **27**

D Reproducibility Pipeline **30**

Abstract

This project investigates the optimisation of OpenMP Cholesky factorisation on CSD3. Starting from a serial baseline, we introduced a sequence of memory access and parallelisation improvements and evaluated their effect across problem sizes and thread counts.

The optimised serial version significantly outperformed the baseline, and OpenMP parallelisation provided further gains for large matrices. The main conclusion is that performance is governed primarily by memory locality and synchronisation overhead, rather than by floating point operation count alone. Additionally, this project stresses the importance of cache behaviour, vectorisation, and parallelisation structures.

1 Mathematical Introduction

A matrix $C \in \mathbb{R}^{n\times n}$ is *symmetric positive definite* (SPD) if and only if there exists a unique lower triangular matrix L with positive diagonal elements such that [1]:

$$C = LL^T. \tag{1}$$

The diagonal elements of L are given by the recurrence relation:

$$L_{pp} = \sqrt{C_{pp} - \sum_{k=0}^{p-1} L_{pk}^2}, \qquad L_{pp} > 0 \text{ (guaranteed by SPD property).} \tag{2}$$

The off-diagonal elements are:

$$L_{ip} = \frac{1}{L_{pp}}\left(C_{ip} - \sum_{k=0}^{p-1} L_{ik}L_{pk}\right), \quad i > p. \tag{3}$$

1.1 Algorithmic structure (Cholesky–Banachiewicz)

The factorisation can be expressed in a right looking (outer-product) form that updates the trailing submatrix $C^{(p+1)}$ at each pivot step $p = 0, 1, \ldots, n-1$:

$$\begin{aligned}
&\text{(i) } L_{pp} \leftarrow \sqrt{C_{pp}^{(p)}}, \text{(ii-a)} && L_{pj} \leftarrow C_{pj}^{(p)}/L_{pp},\ j > p,\\
&\text{(ii-b)} L_{ip} \leftarrow C_{ip}^{(p)}/L_{pp},\ i > p,\\
&\text{(iii) } C_{ij}^{(p+1)} \leftarrow C_{ij}^{(p)} - L_{ip}\, L_{pj}, \quad i, j > p.
\end{aligned} \tag{4}$$

Operation (iii) is the Schur complement update: a rank-1 update on the $(n-p-1)\times(n-p-1)$ trailing submatrix [2, 3].

1.2 Calculation of floating point operations

Total floating point operations (FLOPs):

1. *Square roots*: n operations (negligible).
2. *Row/column scaling*: $\sum_{p=0}^{n-1} 2(n-p-1) = n(n-1) \approx n^2$.
3. *Schur complement update*:

$$\sum_{p=0}^{n-1}(n-p-1)^2 = \frac{(n-1)n(2n-1)}{6} \approx \frac{n^3}{3}.$$

Each element update is a multiply-subtract ($\approx \frac{2}{3}n^3$ scalar FLOPs for the Schur step). Total:

$$W = \frac{n^3}{3} + O(n^2) \approx \frac{n^3}{3} \quad \text{(for large } n\text{).} \tag{5}$$

For $n = 4000$ this is $\approx 2.1 \times 10^{10}$.

1.3 Logarithm of the determinant

Since $\det(C) = \det(L)^2 = \left(\prod_p L_{pp}\right)^2$:

$$\log|\det(C)| = 2\sum_{p=0}^{n-1} \log L_{pp}. \tag{6}$$

This reduces log-determinant computation to $O(n)$ operations from the already computed diagonal. In floating point arithmetic, it is stable up to the expected $O(n\varepsilon)$ rounding error and should be independent of thread count.

2 Implementation

The library exposes one function: `mphil_dis_cholesky(double* c, int n)`. The matrix is a flat row major array ($c[i{\cdot}n + j]$). After a call, the lower triangle contains L and the upper triangle contains L^T. The return value is elapsed wall-clock time $t \geq 0$; error codes are negative (-1.0=invalid input, -2.0=non-SPD pivot).

Performance is reported as

$$\text{GFLOP/s} = \frac{n^3}{3\,t \cdot 10^9},$$

using the estimate $W(n) \approx n^3/3$ from (5).

2.1 Version 0: Serial baseline

$V0$ implements the algorithm as close as possible to equations (4) and serves as the correctness reference. The Schur complement update uses a *j-outer/i-inner* loop:

```
for (int j = p + 1; j < n; ++j) {
  for (int i = p + 1; i < n; ++i) {
    c[i * n + j] -= c[i * n + p] * c[p * n + j];
  }
}
```

Listing 2.1: V0: j-outer/i-inner Schur update

In row major layout the i-inner loop strides by $n \times 8$ bytes per iteration, which is the main performance bottleneck (Section 7.1).

2.2 Version 1: Serial optimised

Five optimisations are applied in this version.

2.2.1 OPT-1: Rearranging the order of loops

In row major storage, $V0$'s j-outer/i-inner loop strides by $n \times 8$ bytes per inner step, so each iteration likely touches a new cache line (64 bytes, holding 8 doubles). Swapping to i-outer/j-inner makes the inner loop stride-1, so most of each loaded cache line is used:

```
for (int i = p + 1; i < n; ++i) {        // i-outer
    Lip = c[i*n+p];                       // scalar load (once)
    for (int j = p + 1; j < n; ++j) {    // j-inner, stride-1
        c[i*n+j] -= Lip * c[p*n+j];
    }
```

```
}
```

Listing 2.2: OPT-1: i-outer/j-inner Schur update

This is the primary reason OPT-1 delivers the largest share of the V1 speedup (Section 7.1).

2.2.2 OPT-2: Hoisting $p{\cdot}n$ and row pointers

Repeated index arithmetic is reduced by computing $p \cdot n$ once per pivot and using row pointers:

```
const int pn = p * n;
double* row_p = c + pn;
...
double* row_i = c + i * n;
```

Listing 2.3: OPT-2: row pointer hoisting

This avoids recomputing $p \cdot n + j$ and $i \cdot n + j$ inside inner loops, reducing integer arithmetic overhead and simplifying the loop structure for the compiler.

2.2.3 OPT-3: Replacing division with multiplication

Division does not vectorise, so there is a vectorisation penalty [4, §4]. Instead of dividing by *diag* many times, we compute one reciprocal:

```
const double inv_diag = 1.0 / diag;    // one division per step
    p
for (int j = p + 1; j < n; ++j)
    row_p[j] *= inv_diag;              // multiplication,
       vectorised
for (int i = p + 1; i < n; ++i)
    c[i * n + p] *= inv_diag;
```

Listing 2.4: OPT-3: reciprocal scaling

At step p we perform 1 division instead of $\approx 2(n-p-1)$, and the scaling loops can now vectorise.

2.2.4 OPT-4: Hoisting L_{ip}

Inside the Schur update, L_{ip} does not depend on j, so it is loaded once per row i:

```
for (int i = p + 1; i < n; ++i) {
    const double Lip = c[i * n + p];    // hoisted outside j-
       loop
    for (int j = p + 1; j < n; ++j) {
        row_i[j] -= Lip * row_p[j];
    }
}
```

Listing 2.5: OPT-4: Lip hoisting

2.2.5 OPT-5: 4-wide i-unrolling

The outer i-loop is unrolled by 4 rows (i, $i{+}1$, $i{+}2$, $i{+}3$), so `row_p[j]` is loaded once and reused for four updates, which increases instruction level parallelism (ILP) by creating four independent update streams in the innermost loop.

2.3 Version 2: OpenMP parallel

2.3.1 Analysis of parallelisation and dependencies

The outer p-loop is sequential: pivot step p uses $C_{pp}^{(p)}$, which depends on the trailing submatrix updates from all previous steps. For a fixed pivot p, the Schur complement update can be parallelised by rows $i = p+1, \ldots, n-1$: each iteration i writes only entries in row i and reads the read-only pivot row p, so the i-rows are independent and can be distributed across threads.

2.3.2 One persistent parallel region

Creating and destroying an OpenMP thread group inside the p-loop would add repeated fork/join overhead. Instead, we create the group once and reuse it for all n pivot steps:

```
#pragma omp parallel default(none) shared(c, n, err)
{
    for (int p = 0; p < n; ++p) {
        #pragma omp single
        {  // pivot check + sqrt + inv_diag + row/col scaling
           (O(n))
        }  // implicit barrier: all threads see row_p before
           Schur
        if (err) break;
        const double* row_p = c + p*n;
        #pragma omp for schedule(static)
        for (int i = p+1; i < n; ++i) {
            double* row_i = c + i*n;
            const double Lip = row_i[p];
            #pragma omp simd
            for (int j = p+1; j < n; ++j)
                row_i[j] -= Lip * row_p[j];
        }  // implicit barrier
    }
}
```

Listing 2.6: V2: persistent parallel region

2.3.3 OpenMP constructions

The following directives are from the OpenMP specification [5].

`#pragma omp parallel default(none) shared(c,n,err)` Forks the thread group once and keeps it alive for all n pivot steps. `default(none)` forces every variable to be explicitly classified. All other variables (loop indices, row pointers, `Lip`) are declared inside the region, so each thread gets its own private copies.

`#pragma omp single` (not `omp master`) Exactly one thread executes the block (pivot check, sqrt, row/column scaling), others wait at the implicit barrier. `omp master` has no implicit barrier, so other threads could enter the Schur update before *row_p* is fully written.

`#pragma omp for schedule(static)` Distributes i-rows of the Schur complement. Work per row is uniform ($n-p-1$ inner-j updates), so static scheduling gives equal-sized contiguous chunks with minimal runtime overhead. The implicit barrier at the end ensures the full Schur update for step p finishes before the next pivot.

`#pragma omp simd` (with `__restrict__`) Requests vectorisation of the innermost j-loop. Iterations are independent; `__restrict__` states that row_i and row_p point to different rows, removing aliasing ambiguity.

2.3.4 Proof of the absence of data races

1. `omp for` splits the i-range, so no two threads write the same $c[i \cdot n + j]$.
2. `row_p` is written only in `omp single`; the implicit barrier guarantees visibility before the Schur update.
3. `err` is written only inside `omp single` and read after the barrier, so all threads break consistently.
4. Variables Lip, row_i, and loop indices are private (declared inside the region or loops).

Therefore the V2 implementation has no data race.

2.4 Version 3: Parallelising the column scaling

In V2, column scaling $c[i \cdot n + p] *=$ inv_diag for $i > p$ runs inside `omp single`, creating a serial stall while other threads wait. In V3, `inv_diag` is stored in a shared variable and column scaling moves into the `omp for` loop: each thread scales its own $row_i[p]$ and uses it immediately as L_{ip}. Pivot row scaling remains in `omp single` because $row_p[j]$ must be fully written before any thread enters the Schur loop.

3 Roadmap of Optimisation Versions

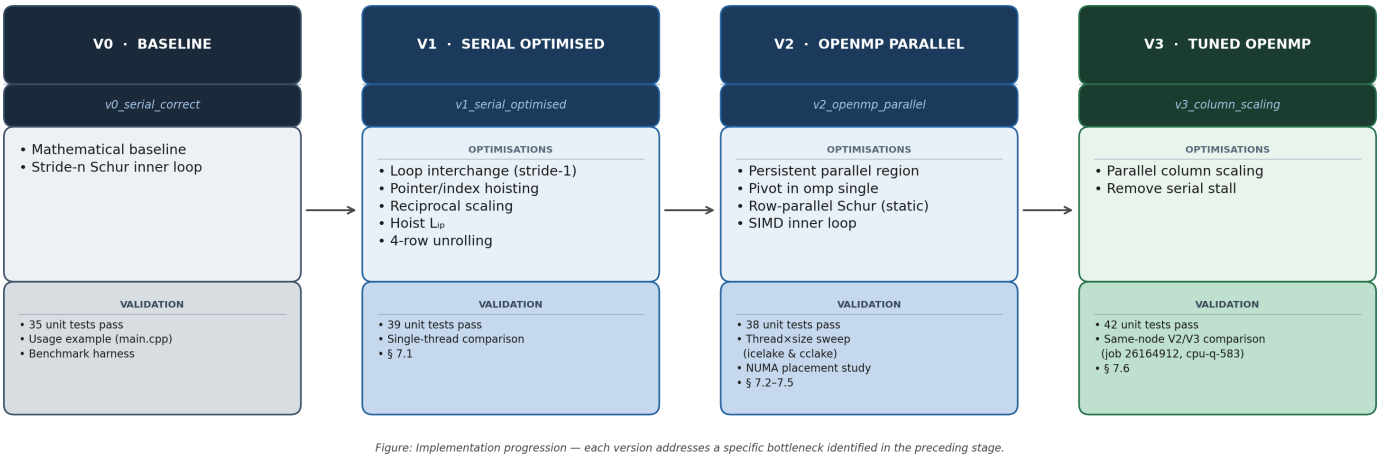

Figure 3.1: Implementation progression from V0 to V3.

4 Test Methodology

Two test executables are built alongside the library:

- **cholesky_test_n2**: runs the example $\mathbf{C} = \begin{bmatrix} 4 & 2 \\ 2 & 26 \end{bmatrix}$ and checks every element against the expected result $\begin{bmatrix} 2 & 1 \\ 1 & 5 \end{bmatrix}$ within tolerance 10^{-12}.
- **cholesky_test**: the full suite, run under **ctest** with `OMP_NUM_THREADS=1` by default, groups requiring multiple thread counts call `omp_set_num_threads()` internally. Test counts: V0-35, V1-39, V2-38, V3-42. All checks use $t \geq 0.0$ because on tiny n inputs `omp_get_wtime()` returns exactly 0.0; error codes -1.0 and -2.0 are unambiguously negative.

The file `src/mphil_dis_cholesky_testing.cpp` has three test only hook categories:

- **Per step**: executes exactly one pivot step p of the factorisation, dispatching on `CHOLESKY_VERSION`. Returns 0 on success or -2 if $c[p \cdot n+p] \leq 0$ (non-SPD).
- **Thread ID (TID)**: during V2/V3 steps, logs `omp_get_thread_num()` for every column-scaling write. Two accessors expose the data; Group H2 uses them to assert that ≥ 2 distinct thread IDs wrote column p, proving parallelism.
- **Write/stride**: captures the first two write addresses inside V1's scalar-tail loop. Address delta 1 confirms j-inner order (OPT-1 active); delta n would indicate V0-style i-inner order. Used by Group I2.

Tests are grouped into 11 groups (A–K)

- **Groups A–F** cover the 2×2 example, closed form matrices, the log-det identity, error return codes, and the symmetry invariant.
- **Group G** checks `mphil_dis_cholesky_step()` against `ref_step()` at every pivot step to verify per step correctness.
- **Groups H and H2** prove the OpenMP implementation: thread invariant logdet rules out data races, a TID recorder test proves $V3$ column scaling actually runs in parallel.
- **Groups I, I2, J** close targeted gaps for $V1$ tail coverage, the loop order verification, and barrier semantics.
- **Group K** rules out a `corr()`-specific implementation using a second SPD family.

Detailed description is in Appendix C.

6 Benchmark Methodology

We benchmarked `mphil_dis_cholesky()` using the deterministic SPD input $\mathrm{corr}(n)$, the same matrix family used in the tests. For each (n, T) setting, we performed one warm up call followed by 3 timed repeats. Before each timed repeat, we restored the input matrix from a precomputed copy because the factorisation overwrites the array in place. We report the upper median of the 3 timings to reduce sensitivity to OS jitter.

Our performance testing strategy was designed to isolate the effect of each optimisation step and to make comparisons reproducible. First, we benchmarked all versions at $T = 1$ in order to separate single thread code improvements from OpenMP scaling effects. This is essential because a performance gain in an OpenMP enabled version does not necessarily imply a parallelism benefit: it may instead come from better compiler optimisation of the serial kernel. Second, for the parallel versions we swept thread count over a broad range, rather than choosing one fixed thread count, because the optimal OpenMP configuration is expected to depend on matrix size. Third, for comparisons intended to attribute a gain to a specific code change, we used same node measurements where possible, since node to node variation on CSD3 can otherwise confound cross version interpretation.

The benchmark protocol also reflects the fact that the factorisation overwrites its input matrix in place. To ensure that each timed run measures the same computation, we restore the input matrix from a precomputed copy before every repeat. A warm-up run is discarded so that one-off effects such as page faults, cache cold-start, or dynamic runtime initialisation do not distort the measured timings. Reporting the median of repeated runs then reduces sensitivity to operating-system jitter and transient system noise. Together, these choices provide a controlled and reproducible basis for evaluating whether an optimisation is genuinely beneficial.

Thread count is controlled via `OMP_NUM_THREADS=T` and SLURM's `-cpus-per-task=T`; the full runtime and pinning configuration is given in Table 7.1.

7 Results and Discussion

Reproducibility metadata. The settings below were used for all results unless otherwise stated.

Item	Value
Code tags	`v0_serial_correct`, `v1_serial_optimised`, `v2_openmp_parallel`, `v3_column_scaling`
CMake flag	`-DCHOLESKY_VERSION=0/1/2/3` `-DCMAKE_BUILD_TYPE=Release`
Platform	CSD3 icelake (76 cores) and cclake (56 cores) [6]
Compiler	GCC 11; `-O3 -march=icelake-server` / `-march=cascadelake`
OpenMP	`-fopenmp` for V2/V3, not linked for V0/V1
Thread control	`OMP_NUM_THREADS=<T>` + `-cpus-per-task=<T>`
Pinning	`OMP_PROC_BIND=close OMP_PLACES=cores`
Input	`corr(n)` (SPD), reset each run, 1 warm up + 3 timed, median

Table 7.1: Reproducibility metadata for all experiments.

GFLOP/s is defined throughout as W/t where $W = n^3/3$ and t is wall-clock time returned by `mphil_dis_cholesky`.

7.1 Single thread: $V0 \to V1 \to V2 \to V3$

Partition: icelake. All versions at $T = 1$, GCC 11 `-O3 -march=icelake-server`.

n	V0	V1	V1/V0	V2 (T=1)	V3 (T=1)	V1/V2
500	1.00	3.79	3.8×	3.77	3.87	1.01
1000	0.727	2.45	3.4×	3.03	3.07	0.81
2000	0.517	0.849	1.6×	3.09	3.14	0.27
4000	0.321	0.729	2.3×	1.84	2.03	0.40
8000	0.154	0.737	4.8×	1.54	1.59	0.48

Table 7.2: GFLOP/s at $T = 1$, all versions, icelake.

The single thread comparison is useful because it separates *serial optimisation effects* from *parallel scaling effects*. In particular, any gap observed at $T = 1$ must come from code generation, memory access, or instruction level behaviour, and not from OpenMP thread level parallelism. This allows the progression $V0 \to V1 \to V2 \to V3$ to be interpreted as a sequence of implementation changes rather than just a set of measured curves.

$V0 \to V1$**: fixing the main locality bottleneck.** V1 applies OPT-1 (loop swap) as the primary change, together with OPT-2–OPT-5 (pointer/index hoisting, reciprocal scaling, hoisting L_{ip}, and 4-wide unrolling).

The dominant optimisation from $V0$ to $V1$ is OPT-1, which changes the loop order of the Schur complement update. In $V0$, this kernel is written in j-outer / i-inner form. Since the matrix is stored in row major order, the inner loop then walks down a column: expressions such as $c[i \cdot n + j]$ and $c[i \cdot n + p]$ are accessed with a stride of $n \times 8$ bytes. For example, at $n = 1000$ this corresponds to an 8000 byte jump between consecutive inner loop iterations. Since a cache line is only 64 bytes, most cache lines brought into cache are only partially used. The result is poor spatial locality and a large amount of wasted memory traffic.

OPT-1 swaps the loop order to i-outer / j-inner, so that the inner loop walks contiguously along a row. In this arrangement, both the updated row entry $c[i \cdot n + j]$ and the pivot row entry $c[p \cdot n + j]$ are accessed with stride-1. A 64 byte cache line then supplies 8 consecutive doubles that are actually used by the inner loop, which is much closer to the access pattern modern caches are designed for. Since the Schur complement is the dominant $O(n^3)$ part of the factorisation, improving its locality has the highest leverage among all serial changes.

This interpretation is consistent with the data. At smaller sizes ($n = 500, 1000$), where the working set is still relatively favourable to cache reuse, $V1$ is faster than $V0$ by about 3.4–3.8×. At $n = 2000$, however, the matrix already occupies about $2000^2 \times 8 \approx 32$ MB, so the computation increasingly depends on moving a large working set through the memory hierarchy. In this regime, improving the loop order still helps, but large data movement is unavoidable, so the relative gain is smaller, correspondingly the measured $V1/V0$ speedup drops to about 1.6× at $n = 2000$. At larger n, the performance gap widens again because the stride-n access pattern in $V0$ becomes increasingly costly relative to the stride-1 kernel in $V1$.

The remaining serial optimisations in $V1$ are still worthwhile, but they are best interpreted as secondary improvements on top of OPT-1. They reduce address arithmetic, redundant loads, and instruction overhead in the hot loop, and can improve instruction scheduling, but they do not alter the main dataflow structure of the kernel. The measured results are therefore consistent with OPT-1 being the main step change, with OPT-2–OPT-5 providing additional but smaller gains.

$V1 \to V2$**: stronger SIMD vectorisation, not parallelism.** A substantial performance gap is already visible at $T = 1$: for $n \geq 2000$, $V1$ is about 2.1–3.6× slower than $V2$. Since both runs use a single thread, this difference cannot be attributed to OpenMP thread-level parallel execution. Instead, it must come from how the compiler is able to optimise the inner Schur-update loop.

In $V2$ (and later in $V3$), the innermost j-loop is written in a form that is more explicitly vectorisable: it is annotated with `#pragma omp simd`, and the row pointers used inside the Schur kernel are marked `__restrict__`. These changes matter because the compiler must be able to prove two things before generating an efficient SIMD loop: first, that iterations over j are independent; second, that the read stream and write stream do not alias. Without a strong no-alias guarantee, GCC may conservatively emit a weaker or guarded loop, especially in a kernel with several pointers and manual unrolling. By using `omp simd` together with `__restrict__`, we explicitly state that the pivot row and the updated row are distinct memory regions and that the loop is safe to vectorise across j.

The measured size dependence supports this interpretation. At $n = 500$, the gap between $V1$ and $V2$ is small, because the total inner loop work is short and the benefit of wide SIMD is limited. At $n = 1000$ the difference becomes more visible, and for $n \geq 2000$ it becomes large, precisely where the Schur update dominates runtime. On `icelake`, this behaviour is consistent with GCC generating a substantially stronger SIMD kernel for the dominant multiply/subtract loop in $V2/V3$ than in $V1$.

This is an inference from code structure and timing behaviour rather than a direct hardware-counter measurement. We did not measure vector instruction counts or FMA usage explicitly. Nevertheless, the fact that the gap appears already at $T = 1$ and grows with n strongly suggests that the main mechanism is compiler-generated SIMD vectorisation rather than any OpenMP runtime effect.

$V2 \to V3$**: no meaningful single thread difference.** At $T = 1$, any difference between $V2$ and $V3$ is at the level of measurement noise rather than a real optimisation effect. This is expected from the implementation: the only change in $V3$ is that column scaling is moved from an `omp single` region into the parallel `omp for` region. With one thread, both versions execute effectively the same arithmetic and the same memory accesses, so there is no reason to expect a systematic speedup. The measured differences of only 0–2% are therefore best interpreted as normal runtime variation.

Overall, the single thread results indicate that the main serial improvements come first from restoring stride-1 locality in the Schur update ($V0 \to V1$), and then from enabling stronger compiler SIMD vectorisation in the same dominant kernel ($V1 \to V2$). By contrast, $V2 \to V3$ has no meaningful effect at $T = 1$, confirming that its benefit is purely parallel-structural rather than serial.

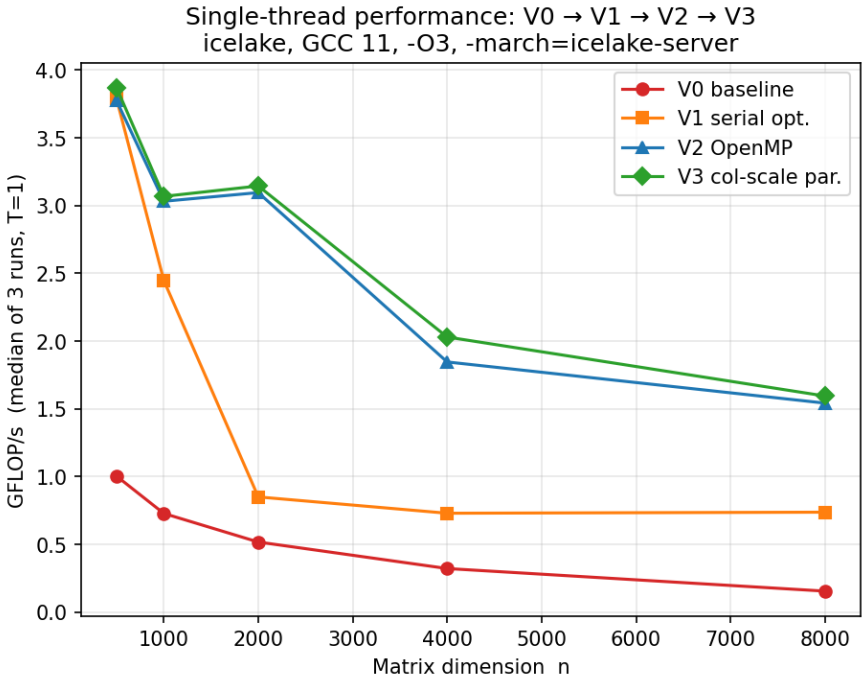

Figure 7.1: Single thread GFLOP/s vs n, V0–V3 on icelake. V2/V3 exceed V1 at large n, consistent with stronger SIMD vectorisation.

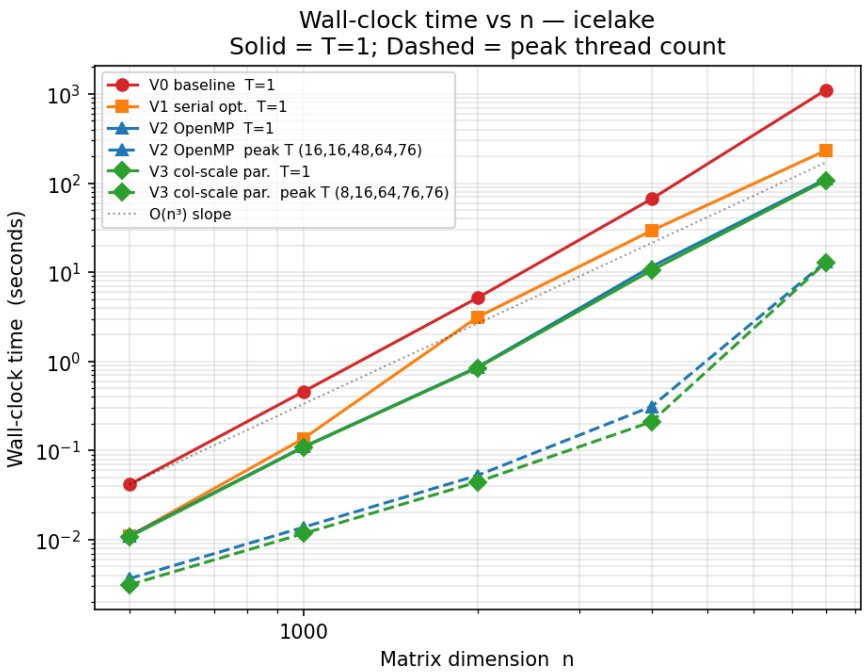

Figure 7.2: Wall-clock time vs n (log–log) for all versions on icelake. All versions follow $O(n^3)$ scaling, the gap between serial and parallel versions grows rapidly with n.

The figures below summarise the same progression visually: first a large single thread gain from improved locality, then a further gain from stronger SIMD code generation, while $V2$ and $V3$ remain essentially identical at $T = 1$.

T	n=500	n=1000	n=2000	n=4000	n=8000
1	3.77	3.03	3.09	1.84	1.54
16	**11.39**	**24.19**	31.10	27.70	10.40
32	8.90	22.35	46.30	46.44	11.61
48	6.77	18.48	**50.67**	59.27	12.24
64	4.49	17.97	44.42	**68.45**	12.64
76	3.22	13.94	35.74	63.50	**13.09**

Table 7.3: V2 GFLOP/s vs thread count, icelake. (bold is peak for each n).

7.2 $V2$ OpenMP scaling (icelake)

Partition: icelake, node cpu-q-536, job 26033045. `CHOLESKY_VERSION=2`, GCC 11 `-O3 -march=icelake-server -fopenmp`.

This validates the benchmarking strategy in Chapter 6: sweeping thread count is necessary, because a fixed T would hide the strong size dependence of the optimal configuration.

Peak: **68.45 GFLOP/s** at $n = 4000$, $T = 64$ (speedup 37.2×, efficiency 58%). Optimal T varies strongly with n, three regimes:

For small matrices, such as $n = 500$, the Schur update itself is relatively cheap: the full matrix is only about 2 MB, so much of the working set can be served efficiently from cache. In that regime, the non-parallel parts of the OpenMP structure become disproportionately important. In $V2$, each pivot step contains one `omp single` region and one `omp for` region, each ending with an implicit barrier, so there are effectively $2n$ barriers per factorisation. At $n = 500$, this means about 1000 barrier synchronisation events. Since the arithmetic per pivot step is small, this fixed overhead becomes a large fraction of total runtime. This explains why performance peaks already around $T = 16$ and then degrades as more threads are added.

For intermediate sizes, especially $n = 2000$ and $n = 4000$, the balance shifts. The Schur complement is an $O(n^3)$ kernel, while the serial work and synchronisation overhead are closer to $O(n)$ or $O(n^2)$ per factorisation. As n grows, the useful parallel work therefore dominates more strongly, so scaling improves. At $n = 2000$, throughput rises up to $T = 48$ and then drops beyond that point, suggesting that the code has moved past its locality friendly region and begins to pay a penalty for increased cross-socket or memory system traffic. At $n = 4000$, the Schur update is much larger again, so the overheads are amortised more effectively, and the best result shifts to $T = 64$, reaching 68.45 GFLOP/s.

For very large matrices, such as $n = 8000$, the behaviour changes again. The matrix occupies about 512 MB, which is far beyond cache capacity, so the Schur update becomes dominated by moving data through the memory system rather than by raw floating-point capability. In that regime, adding threads gives diminishing returns once the available memory throughput is close to saturation. The measured increase from 11.61 GFLOP/s at $T = 32$ to 13.09 GFLOP/s at $T = 76$ is only about 12.7%, despite a much larger increase in thread count. This is characteristic of a bandwidth limited regime rather than a compute limited one.

Amdahl analysis. We use Amdahl's law $S_{\text{Amdahl}}(T) = 1/(s+(1-s)/T)$ to quantify the effective serial fraction s [7]. Per-point estimates from $s = (T/S(T) - 1)/(T - 1)$:

Version	n	T	$S(T)$	s
V2	500	16	3.02×	28.7%
V2	1000	16	7.98×	6.7%
V2	4000	64	37.1×	1.2%
V3	500	8	3.46×	18.8%
V3	1000	16	9.28×	4.8%
V3	4000	64	44.4×	0.7%

Table 7.4: Amdahl serial-fraction estimates from peak speedup.

At small n, s is large because $2n$ barriers and serial `omp single` work are a significant fraction of total time; $S_\infty = 1/s \approx 3.5\times$ matches observed saturation. At $n = 4000$, s drops to 1.2% as $O(n^3)$ Schur work dominates, enabling 37× speedup. V3 reduces s at every size by removing column scaling from the serial region (s drops from 1.2% to 0.7% at $n = 4000$). Note that s bundles explicit serial code, barrier waiting, and bandwidth effects much larger than the naive $O(n)/O(n^3)$ FLOP argument.

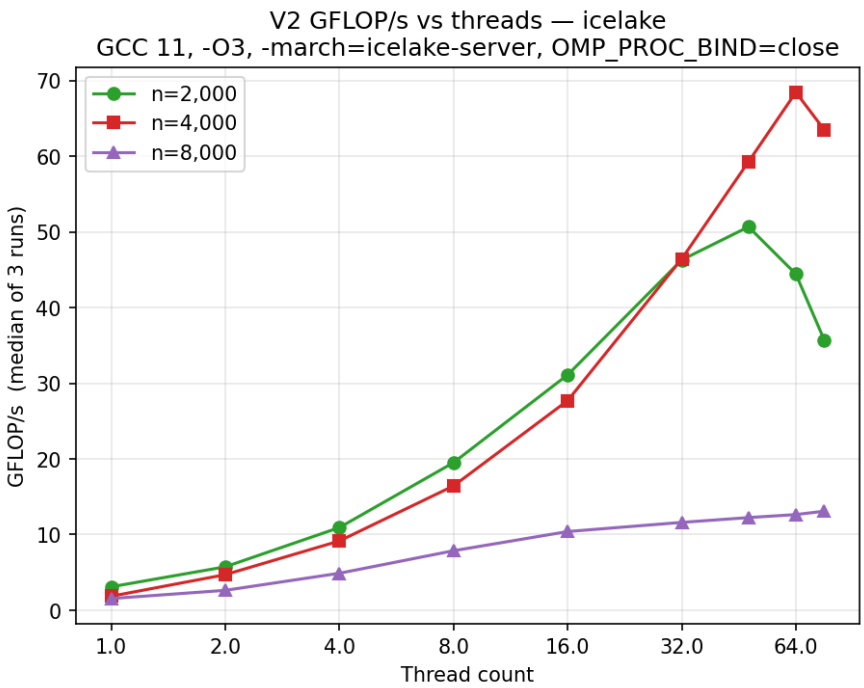

Figure 7.3: V2 GFLOP/s vs T on icelake. $n = 4000$ scales best (peak 68.45 GFLOP/s at $T = 64$); $n = 2000$ drops after $T = 48$; $n = 8000$ flattens beyond $T \approx 32$.

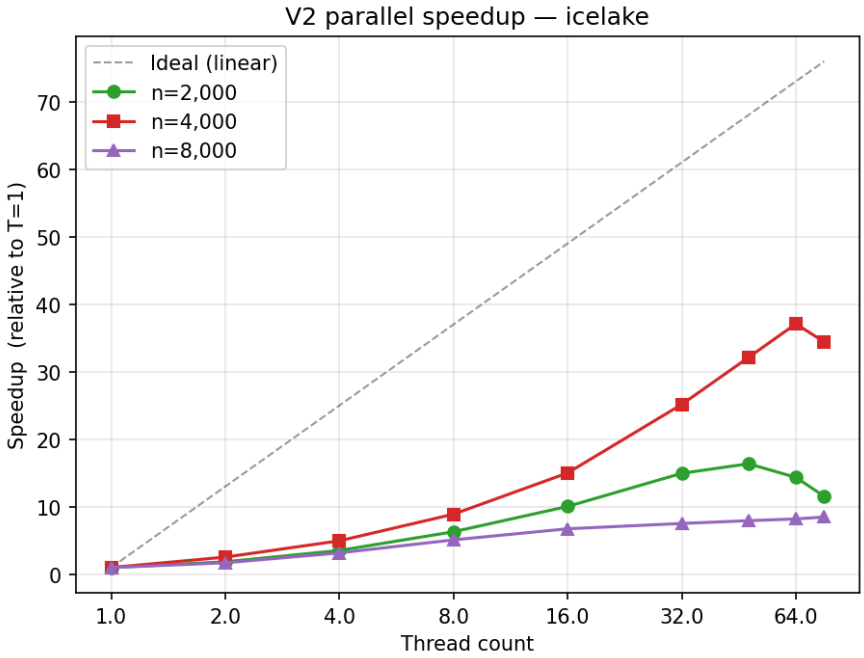

Figure 7.4: Parallel speedup $S(T)$ vs T for V2 on icelake. Best scaling at $n = 4000$ ($S(64) = 37.2\times$); dashed line is ideal linear speedup.

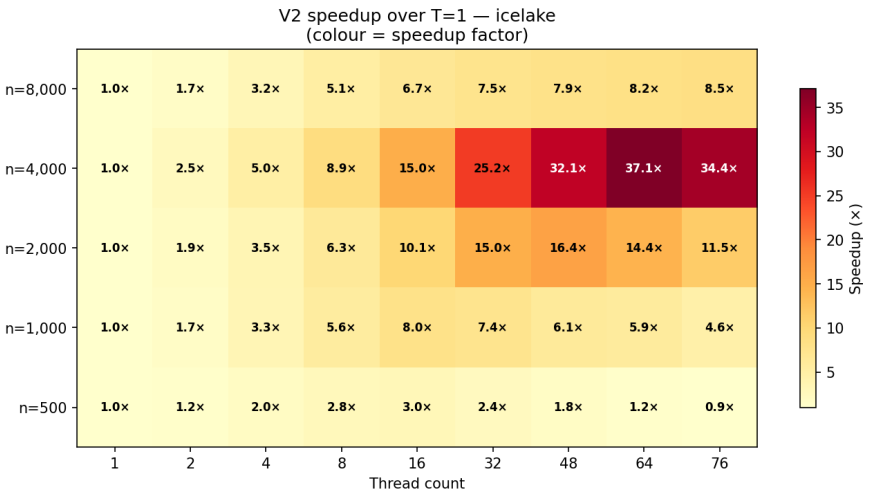

Figure 7.5: $S(T)$ heatmap over all (n, T) combinations, V2 icelake. Bottom-right (small n, high T) shows $S < 1$; top-right reaches $\approx 37\times$.

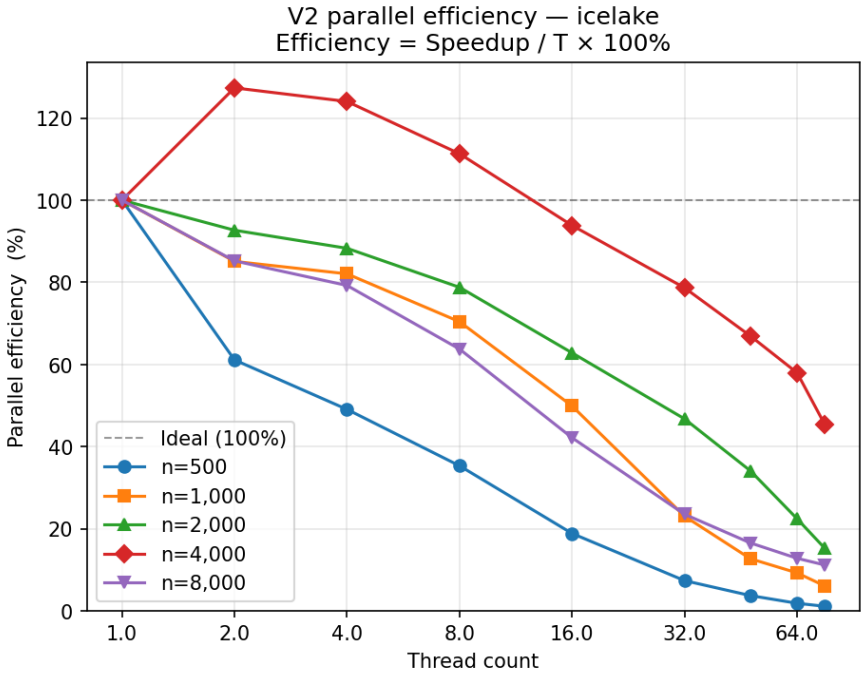

Figure 7.6: Parallel efficiency $E(T) = S(T)/T \times 100\%$ vs T, V2 icelake. At $n = 500$ efficiency drops sharply, at $n = 4000$ it remains $\approx 58\%$ at $T = 64$.

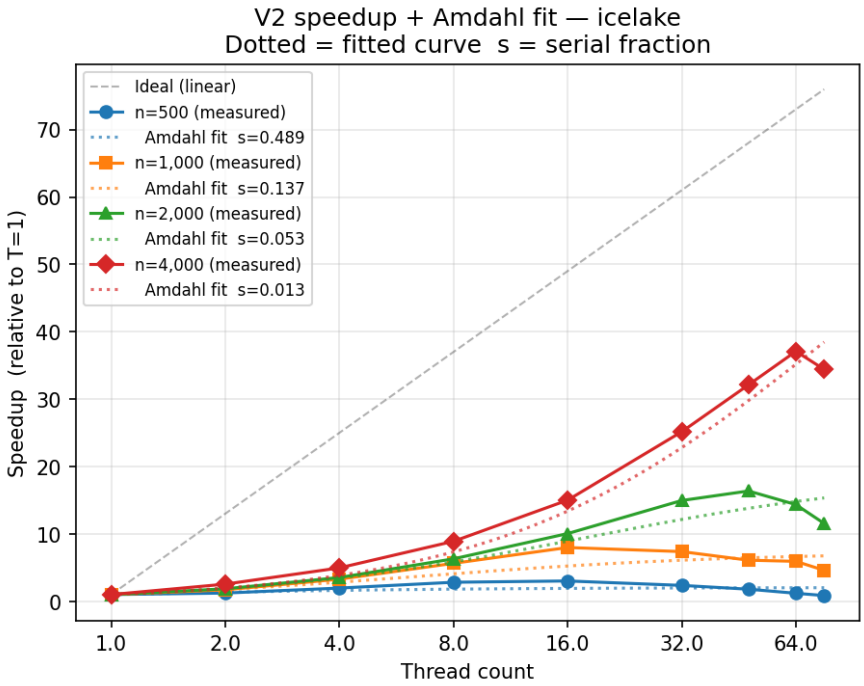

Figure 7.7: Measured $S(T)$ with fitted Amdahl curves, icelake. Larger $n \to$ smaller $s \to$ better scaling.

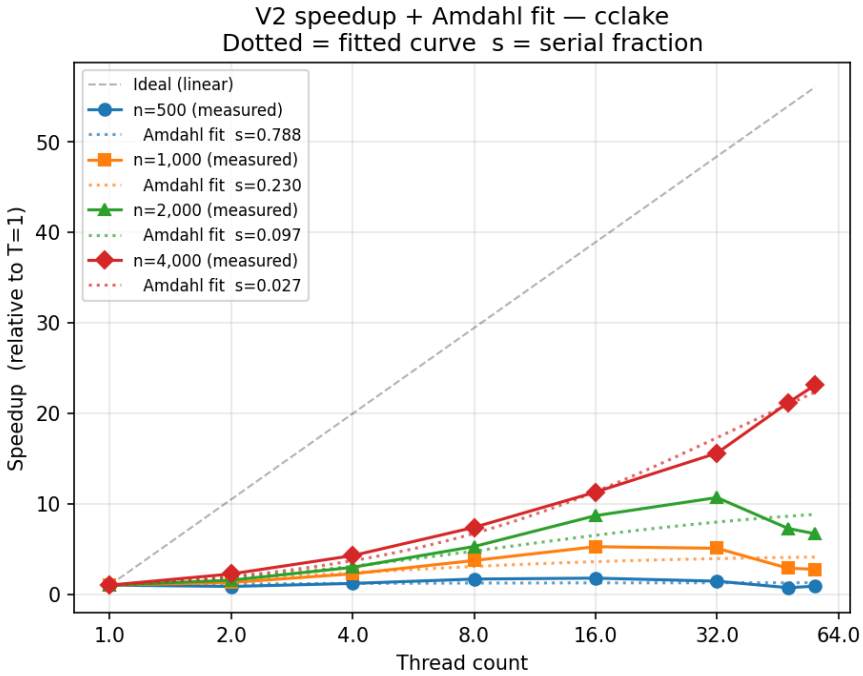

Figure 7.8: Amdahl fit for V2 on cclake. Qualitative trend matches icelake; fitted s values generally larger, reflecting higher overheads.

7.3 V2 OpenMP scaling (cclake)

Cclake node cpu-p-163, GCC 11 `-O3-march=cascadelake`, $T \in \{1, 2, 4, 8, 16, 32, 48, 56\}$:

T	n=500	n=1000	n=2000	n=4000	n=8000
1	3.41	2.93	2.91	1.59	1.45
8	5.71	10.92	15.31	11.72	6.04
16	**6.05**	**15.38**	25.22	17.91	**6.55**
32	4.91	14.87	**31.03**	24.75	6.50
48	2.44	8.42	21.12	33.59	6.46
56	3.00	8.07	19.40	**36.68**	6.56

Table 7.5: V2 GFLOP/s vs thread count, cclake. (bold is peak for each n).

Peak: **36.68 GFLOP/s** at $n = 4000$, $T = 56$ (23.1×). Qualitative trends match icelake: synchronisation limits small n; $n = 2000$ drops above $T = 32$ from cross socket reads of *row_p* (Section 7.5); $n = 8000$ is bandwidth limited (Section 7.4). Absolute peaks are lower than icelake, consistent with lower per-socket theoretical peak bandwidth (≈ 141 vs ≈ 204 GB/s [6]) and fewer cores (56 vs 76).

The `cclake` results are qualitatively similar to `icelake`, but they are consistently lower in absolute throughput and somewhat earlier to saturate. This is important because it shows that the optimisation trends are not specific to one node type: the same implementation changes improve performance in the same order, but the platform determines where the scaling limit is reached.

At small sizes ($n = 500, 1000$), the best performance is already reached around $T = 16$, after which additional threads provide little benefit or even reduce throughput. This matches the same synchronisation-dominated picture as on `icelake`. At $n = 2000$, the curve peaks at $T = 32$ and then drops sharply at $T = 48$ and $T = 56$. The most consistent interpretation is that this size is large enough to benefit from one socket, but not large enough for the extra threads on the second socket to compensate for the increased cost of repeatedly reading the shared pivot row across sockets. This interpretation is later supported directly by the dedicated NUMA placement experiment in Section 7.5.

At $n = 4000$, the larger $O(n^3)$ workload makes it worthwhile to use both sockets, and the best result shifts to $T = 56$, which is the maximum tested thread count on `cclake`. At $n = 8000$, however, the results are almost flat from $T = 16$ onwards, which is again consistent with a bandwidth-limited regime. In other words, `cclake` shows the same three regimes as `icelake`, but reaches the bandwidth/socket limit earlier and at a lower peak throughput.

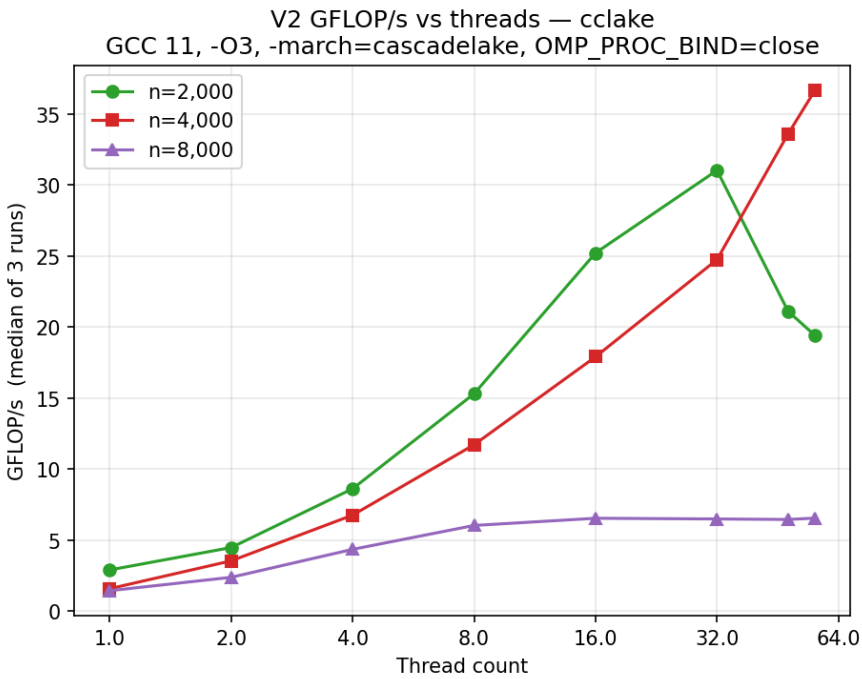

Figure 7.9: V2 GFLOP/s vs T on cclake. Peak 36.68 GFLOP/s at $n = 4000$, $T = 56$; $n = 2000$ drops above $T = 32$; $n = 8000$ is flat from $T = 16$.

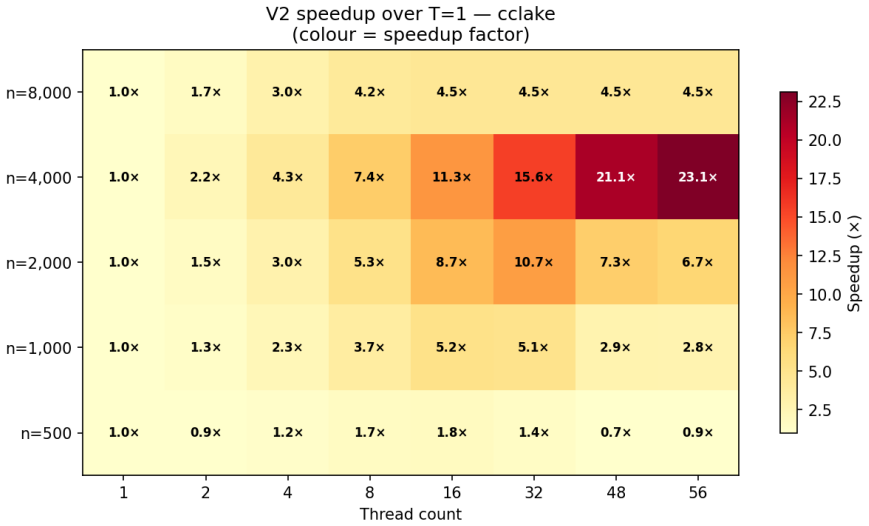

Figure 7.10: $S(T)$ heatmap, V2 cclake. Peak 23.1× at $n = 4000$, $T = 56$.

7.4 Partition comparison and bandwidth saturation

At $n = 4000$: icelake peaks at 68.45 GFLOP/s ($T = 64$) vs cclake 36.68 GFLOP/s ($T = 56$): 1.87× gap. At $n = 8000$: 13.09 vs 6.56 GFLOP/s (2.0×). These ratios are consistent with the per-socket theoretical peak bandwidth (≈ 204 vs ≈ 141 GB/s [8]) and the higher core count on icelake (76 vs 56).

Memory bandwidth saturation. Using the roofline model [9], the Schur update arithmetic intensity is $AI \approx 2/24 = 0.083$ FLOP/B (1 FMA per element: 8 B read $row_p[j]$, 8 B read and 8 B write $row_i[j]$), well below typical balance points. The per-socket theoretical peak gives a roofline ceiling of $0.083 \times 204 \approx 17$ GFLOP/s (icelake) and $0.083 \times 141 \approx 12$ GFLOP/s (cclake). Observed 13.09 and 6.56 GFLOP/s at $n = 8000$ are below these ceilings, confirming bandwidth-limited behaviour; the kernel cannot saturate the full dual socket aggregate because the pivot row broadcast crosses the UPI inter socket link (Section 7.5). The 2× inter platform gap is consistent with the bandwidth ratio [10].

The platform comparison also helps separate *algorithmic* conclusions from *machine-specific* ones. The algorithmic conclusion is that the Cholesky Schur-update kernel benefits from locality-aware serial optimisation and then from OpenMP row wise parallelism.

The machine specific conclusion is that the final throughput depends strongly on the memory system: `icelake` sustains higher throughput than `cclake`, especially at larger n, because the kernel is increasingly constrained by memory bandwidth and socket level data movement rather than by scalar floating point capability alone. This is why the inter-platform gap is modest at low thread counts but becomes much larger once the code enters the high throughput parallel regime.

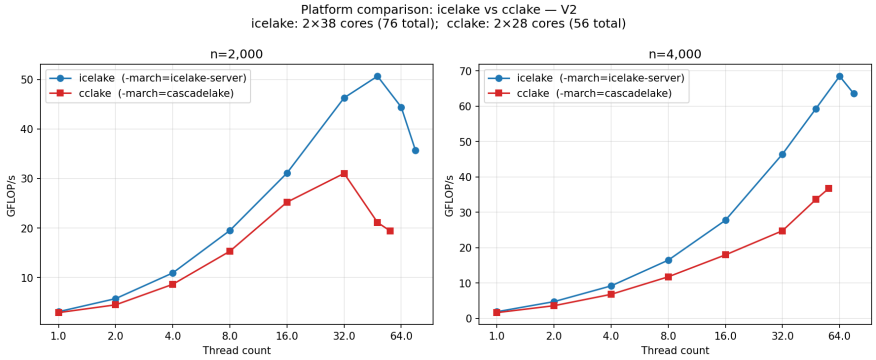

Figure 7.11: V2 GFLOP/s vs T for icelake (blue) and cclake (red) at $n = 2000$ and $n = 4000$. icelake sustains higher throughput, consistent with its higher bandwidth and core count.

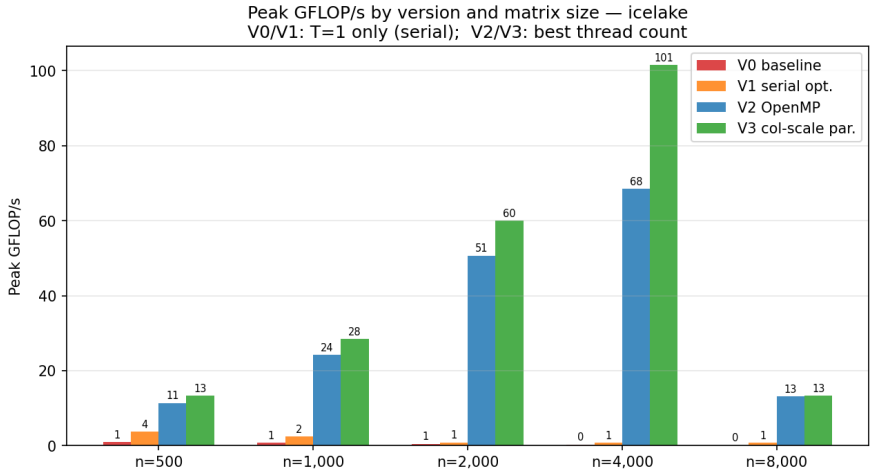

Figure 7.12: Peak GFLOP/s over all thread counts on icelake, all versions. V3 uses same node data for $n \in \{2000, 4000\}$ to reduce node to node variation.

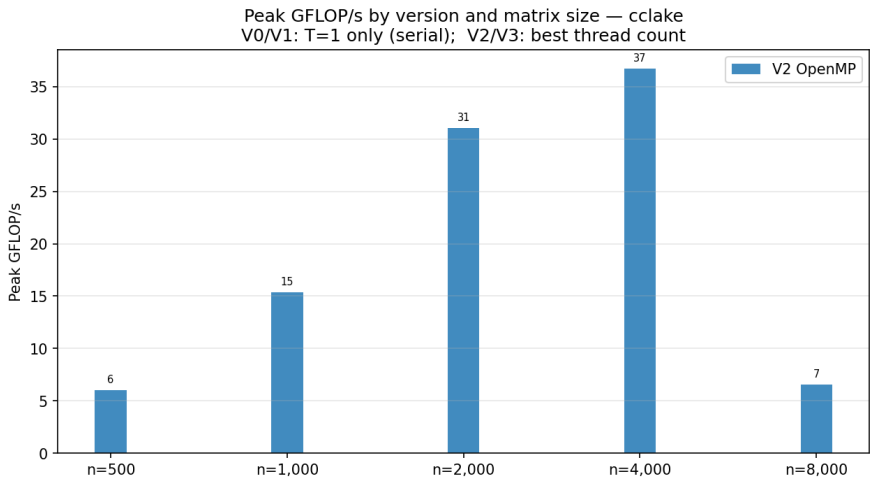

Figure 7.13: Peak GFLOP/s on cclake (V2). Lower than icelake at large n due to lower bandwidth.

7.5 Non-Uniform Memory Access experiment

Cclake node cpu-p-134, V2, $n = 2000$; only `OMP_PROC_BIND` varied:

T	close (GFLOP/s)	spread (GFLOP/s)	close/spread
4	8.37	10.52	0.80
8	15.25	15.37	≈ 1.0
16	24.78	21.35	1.16
32	**30.89**	23.85	1.29
56	22.87	12.54	**1.82**

Table 7.6: GFLOP/s with `OMP_PROC_BIND=close` vs `spread`, $n = 2000$, cclake.

`close` outperforms `spread` from $T \geq 16$, reaching 1.82× at $T = 56$. The Schur inner loop reads the same pivot row row_p ($\approx$ 16 KB at $n = 2000$) for every updated row i, a broadcast/read pattern characteristic of NUMA sensitive kernels. With `close`, threads stay on one socket longer, keeping row_p hot in local cache; with `spread`, a larger fraction fetch row_p via the inter-socket link. The exception at $T = 4$ (spread wins by 1.25×) occurs because at low thread counts remote-read pressure is small and spreading improves aggregate write bandwidth. **Use `OMP_PROC_BIND=close` `OMP_PLACES=cores` for this kernel on dual-socket nodes.**

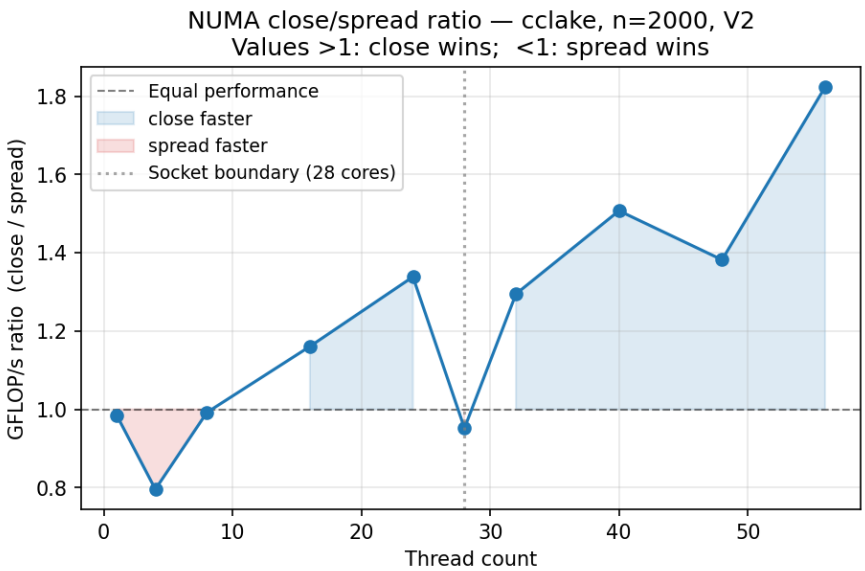

Figure 7.14: Ratio GFLOP/s(close)/GFLOP/s(spread) vs T at $n = 2000$, cclake. Ratio crosses 1.0 between $T = 8$ and $T = 16$; reaches 1.82 at $T = 56$.

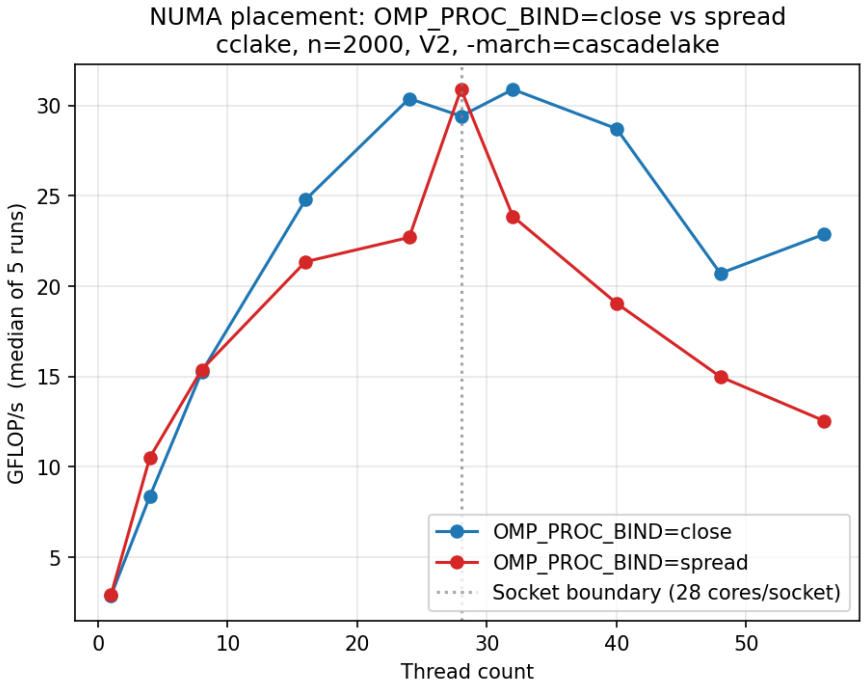

Figure 7.15: Absolute GFLOP/s for close and spread vs T, $n = 2000$, cclake. `close` peaks at $T = 32$ (30.89 GFLOP/s); `spread` drops to 12.54 GFLOP/s at $T = 56$.

7.6 Same node V2 vs V3 comparison

Both versions benchmarked in one SLURM allocation on the same node, 5 repeats per configuration, median reported. Matrix sizes $n \in \{2000, 4000\}$; thread counts $T \in \{1, 32, 48, 64, 76\}$.

T	V2 n=2000	V3 n=2000	Δ%	V2 n=4000	V3 n=4000	Δ%
1	3.11	3.15	+1	2.04	2.04	0
32	46.45	65.38	**+41**	45.91	51.82	+13
48	48.57	50.20	+3	59.81	70.62	+18
64	42.84	51.92	+21	72.03	**90.48**	**+26**
76	46.47	55.08	+19	71.79	89.97	+25

Table 7.7: Same node V2 vs V3 GFLOP/s comparison on icelake.

V3 is +41% at ($n = 2000$, $T = 32$) and +26% at ($n = 4000$, $T = 64$); at $T = 1$ the difference is ≤ 1%. The gain is disproportionate to the $O(n^2)/O(n^3)$ FLOP share of column scaling because the bottleneck is memory latency: in V2 one thread walks the column with stride $n \times 8$ bytes while the other $T-1$ threads idle at the barrier. V3 distributes this work across threads in `omp for`, each thread scaling its own row and using the result immediately as L_{ip}. The gain is proportionally larger at $n = 2000$ than $n = 4000$ because the serial fraction scales as $O(n^2)$ while the Schur work scales as $O(n^3)$.

The non-monotone behaviour at $n = 2000$ (V2/V3 both drop between $T = 32$ and $T = 48$) reflects increasing barrier-synchronisation overhead relative to the shrinking Schur update as T grows beyond the bandwidth optimum for that matrix size. At $n = 4000$ the optimal thread count shifts to $T = 64$, consistent with the larger working set filling more of the 76-core memory bandwidth.

The $V2 \to V3$ comparison is especially instructive because it shows that a relatively small amount of arithmetic can still have a large effect on end to end performance if it sits in the wrong place in the parallel structure. In operation-count terms, column scaling is only an $O(n^2)$ part of the factorisation, whereas the Schur complement is $O(n^3)$. A naive FLOP-count argument would therefore predict only a very small overall benefit from optimising column scaling. However, this is not what is observed: on the same node, $V3$ is faster than $V2$ by as much as +41% at $(n = 2000, T = 32)$ and by about +26% at $(n = 4000, T = 64)$.

The reason is that in $V2$, column scaling is placed inside the `omp single` region. This means that one thread performs the entire column walk while all other threads wait at the barrier before the parallel Schur update can begin. In row major storage, walking down a column corresponds to a stride of $n \times 8$ bytes. For $n = 4000$, this is a 32 KB jump per step, so consecutive accesses are typically on different cache lines and the loop behaves like a latency heavy scan with poor spatial locality. The cost is therefore not just its arithmetic count, but the fact that it forms a serial memory latency stall on the critical path of every pivot step.

In $V3$, this work is moved into the `omp for` region: each thread scales its own $row_i[p]$ and then immediately reuses that value as L_{ip} in the Schur update for the same row. This has two benefits. First, the serial stall is removed, so the thread team no longer idles while one thread performs the whole column walk. Second, the scaled value is consumed immediately by the same thread, giving a more local producer–consumer pattern than the full serial column scan in $V2$.

The relative effect is larger at $n = 2000$ than at $n = 4000$, which is also consistent with the cost model. The serial column-scaling contribution grows like $O(n^2)$, while the Schur work grows like $O(n^3)$. As n increases, the same serial stall becomes a smaller fraction of total runtime, so the proportional benefit of removing it decreases. The data therefore supports the interpretation that $V3$ is not reducing the total FLOP count in an important way; instead, it is removing a badly placed serial memory-latency bottleneck from the parallel execution path.

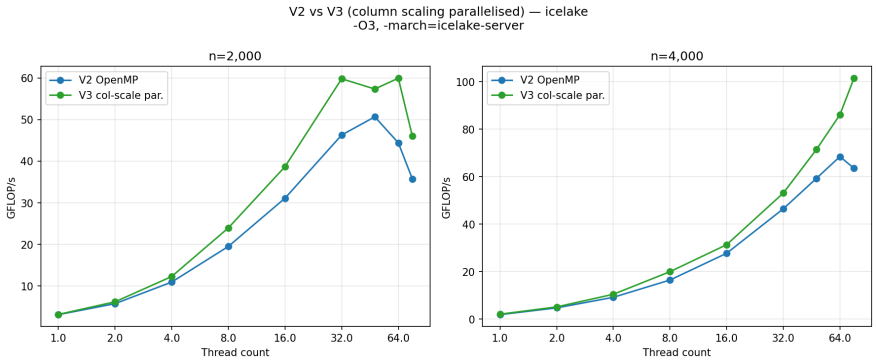

Figure 7.16: GFLOP/s for V2 and V3 vs T at $n \in \{2000, 4000\}$, same node. V3 gap widens with T and is proportionally larger at $n = 2000$.

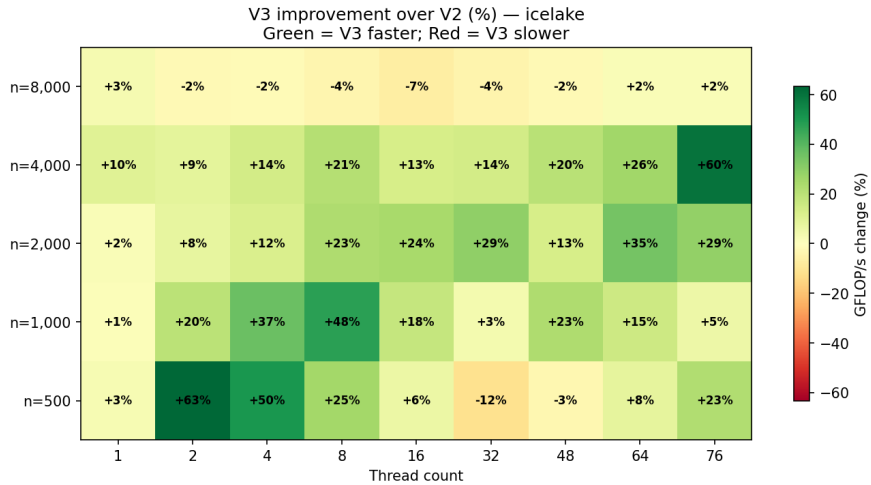

Figure 7.17: V3/V2 improvement heatmap over (n, T). Gain peaks at +41% for $(n = 2000, T = 32)$; near zero at $T = 1$ and $n = 8000$.

8 Conclusions

The peak GFLOP/s bar charts (Section 7.4) summarise the optimisation path $V0 \to V3$ on icelake.
Improvements: $V0 \to V1$ gives up to 3.8× at small n by restoring stride-1 cache access, dropping to 1.6× at $n = 2000$ as the 32 MB matrix becomes memory-bound. $V1 \to V2$ adds 2.1–3.6× at $T = 1$ from stronger SIMD vectorisation, then further gains from parallelism. $V2 \to V3$ adds up to 41% at mid n by removing a serial stall. The final implementation (V3) reaches **90.48 GFLOP/s** at $n = 4000$, $T = 64$ on icelake. At $n = 8000$ all versions are bandwidth limited.

The main lesson is that performance is determined by *memory access patterns* and *synchronisation* rather than by FLOP count.

Taken together, the results show a coherent optimisation story rather than a collection of independent measurements. The serial changes first improved the memory access pattern of the dominant kernel; the OpenMP version then exposed thread-level parallelism in that kernel; and the final tuning step removed a residual serial stall that had remained on the critical path. The figures and tables across Sections 7.1–7.6 therefore provide evidence not only that performance improved, but also *why* each optimisation improved it and under which matrix-size/thread-count regimes that improvement was most visible.

1. **V0 (baseline).** Correctness reference. Main bottleneck: stride-n access in the Schur update.
2. **V1 (serial optimisations).** OPT-1 (loop-order swap) restores stride-1 access and gives the dominant gain. OPT-2–5 add smaller improvements via hoisting, reciprocal scaling, and unrolling.
3. **V2 (OpenMP parallel).** Reaches 68.45 GFLOP/s at $n = 4000$, $T = 64$ (37×, 58% efficiency). Optimal T is strongly n-dependent: barriers dominate at small n, bandwidth dominates at large n, use measured curves rather than a fixed T.
4. **V3 (tuning).** Parallelising column scaling removes a serial memory-latency stall, adding up to +41% at mid n (+26% at $n = 4000$, $T = 64$). Peak: 90.48 GFLOP/s at $n = 4000$, $T = 64$.

NUMA: `OMP_PROC_BIND=close` outperforms `spread` by up to 1.82× at $T = 56$ (Section 7.5) due to the broadcast/read pattern of the pivot row.

Bibliography

[1] Wikipedia. Razlozheniye kholeckogo (Cholesky decomposition), 2024. Accessed March 2026.

[2] V. M. Verzhbitsky. *Fundamentals of Numerical Methods*. Vysshaya Shkola, Moscow, 2009.

[3] Wikipedia. Cholesky decomposition, 2024. Accessed March 2026.

[4] James Fergusson. High performance computing. MPhil in Data Intensive Science lecture notes, University of Cambridge, 2025.

[5] OpenMP Architecture Review Board. OpenMP application programming interface, version 5.2. Technical report, OpenMP ARB, November 2021.

[6] Cambridge Research Computing. Cambridge service for data driven discovery (CSD3), 2024. Accessed March 2026.

[7] Gene M. Amdahl. Validity of the single-processor approach to achieving large-scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485. ACM, 1967.

[8] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation, 2024.

[9] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for floating-point programs and multiprocessors. *Communications of the ACM*, 52(4):65–76, 2009.

[10] John D. McCalpin. STREAM: Sustainable memory bandwidth in high performance computers. Technical report, University of Virginia, 1991. Continuously updated; accessed 2026. `http://www.cs.virginia.edu/stream/`.

A Repository Structure

```
c2_cholesky/— project root
|- include/
|   |- mphil_dis_cholesky.h— public API and function signature
|- src/
|   |- mphil_dis_cholesky.cpp— V0–V3 algorithm implementations
|   |- mphil_dis_cholesky_testing.cpp— test hooks)
|- test/
|   |- test_n2.cpp— minimal 2x2 test
|   |- test_cholesky.cpp— full test suite, groups A–K
|- benchmark/
|   |- benchmark.cpp— CSV benchmark driver
|- example/
|   |- main.cpp— usage demonstration
|- jobs/— CSD3 SLURM job scripts (icelake, cclake)
|- scripts/
|   |- plot_bench.py— generate all benchmark figures
|- results/— raw benchmark CSV output files
|- report/
|   |- report.pdf— this document
|   |- figures/— generated PNG figures
|- CMakeLists.txt— CMake build configuration
|- README.md— build, run, and usage instructions
```

Target	Type	Links	Purpose
`cholesky_lib`	static lib	—	Production library
`cholesky_testing_hooks`	static lib	`cholesky_lib`	Test only hooks
`cholesky_example`	executable	`cholesky_lib`	Usage demo
`cholesky_test_n2`	executable	`cholesky_testing_hooks`	Smoke test
`cholesky_test`	executable	`cholesky_testing_hooks`	Full suite (A–K)
`cholesky_benchmark`	executable	`cholesky_lib`	CSV benchmark

Table A.1: CMake targets.

B Tagged Commits

Tag	Description
`v0_tests`	Initial 27 test suite
`v0_1_tests`	Basic example (n=2)
`v0_serial_correct`	Version 0 baseline implementation
`v0_example`	Example program with corr() matrix
`v1_serial_optimised`	Version 1: five serial optimisations
`v2_openmp_parallel`	Version 2: OpenMP persistent region and SIMD
`v3_column_scaling`	Version 3: column scaling parallelised

Table B.1: Tagged commits.

C Build and Tests

Local build:

```
cmake -S . -B build -DCHOLESKY_VERSION=N -DCMAKE_BUILD_TYPE=
   Release
cmake --build build -j4
ctest --test-dir build --output-on-failure
```

Listing C.1: Local build

Build all four versions:

```
for V in 0 1 2 3; do
    cmake -S . -B build_v$V -DCHOLESKY_VERSION=$V \
          -DCMAKE_BUILD_TYPE=Release
    cmake --build build_v$V -j4
    ctest --test-dir build_v$V --output-on-failure
done
```

Listing C.2: Build all versions

CSD3 benchmark jobs (from repo root):

```
mkdir -p logs results
sbatch jobs/bench_icelake_v0.sh
sbatch jobs/bench_icelake_v1.sh
sbatch jobs/bench_icelake.sh
sbatch jobs/bench_icelake_v3.sh
sbatch jobs/bench_cclake.sh
sbatch jobs/bench_cclake_numa.sh
sbatch jobs/compare_v2v3_icelake.sh
```

Listing C.3: SLURM job submission

Generate figures and tables:

```
python3 scripts/plot_bench.py --results results/ --outdir
   report/
```

Listing C.4: Plot generation

Test groups:

Table C.1: Test group summary (A-K). 'Chk' is check count under `CHOLESKY_VERSION=3`.

Group	Function(s)	Ver.	Chk	Property
A - Example	`test_spec_2x2`	all	2	2×2 example: stored array equals `[2, 1, 1, 5]` within 10^{-14}. Ground truth case from the spec; any implementation error shows up immediately. Return value $t \geq 0.0$ checked separately.
B - Analytical	`test_scalar_n1`, `test_identity`, `test_scaled_identity`	all	8	$n = 1$ scalar ($\sqrt{4} = 2$, tol 10^{-14}); identity $\mathbf{I}_n$ (diagonal $\equiv 1$, logdet $\equiv 0$); $k\mathbf{I}_n$ (diagonal $= \sqrt{k}$, logdet $= n \log k$). Catches sign errors, off-diagonal contamination, and wrong diagonal scaling.
C - Backward error	`test_corr_backward`	all	5	$\lVert\mathbf{C} - \hat{\mathbf{L}}\hat{\mathbf{L}}^\top\rVert_F / \lVert\mathbf{C}\rVert_F \leq$ (10 or 20) $n\varepsilon$ for `corr()` matrices at $n \in \{5, 16, 64, 128, 200\}$. Uses only the lower triangle; symmetry bugs are covered by Group F.
D - Logdet identity	`test_logdet_2x2_analytical`, `test_logdet_corr_sign`	all	2	$\log\lvert\det \mathbf{C}\rvert = 2\sum_p \log L_{pp}$. Exact check against $\log(100)$ (tol 10^{-13}) and against an inline reference within $n \cdot 10^{-12}$.
E - Invalid inputs	`test_invalid_inputs`, `test_non_spd`	all	6	Null pointer, $n \leq 0$, $n > 10^5$ each return -1.0. Non-SPD matrix returns -2.0.
F - Structural	`test_structure`	all	4	Lower–upper symmetry: $c[i \cdot n + j] = c[j \cdot n + i]$ within 10^{-13} for all $i \neq j$; $L_{pp} > 0$ for all p. Complements Group C: a wrong mirroring direction would pass C but fail here.

(continued on next page)

(Table C.1 continued from previous page)

Group	Function(s)	Ver.	Chk	Property
G - Step oracle	`test_step_level`	all	2	Per-pivot comparison of `mphil_dis_cholesky_step()` against `ref_step()` for $n \in \{5, 16\}$; tolerance $50\,n\varepsilon\,\|\mathbf{C}\|_\infty$ after every step. Catches subtle one-step bugs that cancel in the end-to-end result.
H - Thread invariance	`test_openmp_thread_invariance`	V2, V3	2	Backward error and logdet at $T \in \{1, 2, 4, 8, 16\}$: logdet deviation from $T = 1$ must be $\leq 10^{-10}$. Confirms no data races and no thread-count-dependent numerical results.
H2 - V3 col scaling parallel	`test_v3_col_scaling_parallel`	V3	4	At $n = 256$, $T = 8$, $p = 0$: inspects recorded thread IDs and asserts ≥ 2 distinct IDs wrote column $p = 0$. Proves column scaling is actually parallel in V3, distinguishing it from V2.
I - V1 tail coverage	`test_v1_tail_coverage`	V1	2	`mphil_dis_cholesky_step()` at $n \in \{6, 7\}$, $p = 0$, checking scalar-tail rows against `ref_step()`. A wrong loop bound in OPT-5 silently skips tail rows; Groups C–G do not expose this.
I2 - V1 OPT-1 structural	`test_v1_opt1_structural`	V1	2	Calls `mphil_v1_schur_write_stride()` and asserts the address delta between consecutive writes is 1 (j-inner, OPT-1 active), not n (i-inner, V0 order). Only nontiming proof that OPT-1 is operative.
J - Barrier sanity	`test_barrier_sanity`	V2, V3	1	At $T = 8$, $n = 200$, $p = 0$ on a 100×-scaled `corr` matrix: a missing barrier after `omp single` would produce Schur errors ≈ 4.5 per element, twelve orders of magnitude above tolerance $50\,n\varepsilon$.

(continued on next page)

(Table C.1 continued from previous page)

Group	Function(s)	Ver.	Chk	Property
K - SPD stress	`test_spd_stress`	all	6	Backward error and logdet for $\mathbf{C} = \mathbf{A}\mathbf{A}^\top + 0.1\mathbf{I}$ ($A_{ij} = \sin((i{+}1)(j{+}1))/\sqrt{n}$) at $n \in \{8, 40\}$. Confirms correctness is not restricted to the `corr()` family.

D Reproducibility Pipeline

Section	CSV file	Job ID / Node
§7.1 V0 ($n \leq 4000$)	`bench_icelake_v0_26153761...`	26153761 / cpu-q-225
§7.1 V0 ($n = 8000$)	`bench_icelake_v0_n8000_26458876...`	26458876 / cpu-q-142
§7.1 V1	`bench_icelake_v1_26090357...`	26090357 / cpu-q-210
§7.2 V2 icelake	`bench_icelake_26033045...`	26033045 / cpu-q-536
§7.3 V2 cclake	`bench_cclake_26033046...`	26033046 / cpu-p-163
§7.5 NUMA close	`bench_cclake_numa_close_26093275...`	26093275 / cpu-p-134
§7.5 NUMA spread	`bench_cclake_numa_spread_26093275...`	26093275 / cpu-p-134
§7.6 V2 same node	`compare_v2_26164912_cpu-q-583.csv`	26164912 / cpu-q-583
§7.6 V3 same node	`compare_v3_26164912_cpu-q-583.csv`	26164912 / cpu-q-583

Table D.1: Experiment ledger: result sections mapped to source CSV and job ID.